

Review: Incomplete Answers

Let me grade what you submitted:

Question 1: CORRECT

Your Answer: "4 t ff"

Correct. A 4-bit counter needs **4 T flip-flops** (one per bit).

Question 2: INCOMPLETE

Your Answer: "t"

This doesn't answer the question. I asked: **What should t2 be?**

Let me clarify what I'm asking:

In your 2-bit counter, you had:

```
assign t0 = 1'b1;      // Bit 0: always toggle
assign t1 = q[0];      // Bit 1: toggle when q[0]=1
```

For a 4-bit counter, you'd have:

```
assign t0 = 1'b1;      // Bit 0: always toggle
assign t1 = q[0];      // Bit 1: toggle when q[0]=1
assign t2 = ???;       // Bit 2: toggle when ???
assign t3 = ???;       // Bit 3: toggle when ???
```

I'm asking you to fill in the ??? for t2.

Hint: Look at the Counting Pattern

Count | q3 q2 q1 q0 | When does q2 toggle?

```
-----|-----|-----
0   | 0  0  0  0 |
1   | 0  0  0  1 |
2   | 0  0  1  0 |
3   | 0  0  1  1 | q2 stays 1
4   | 0  1  0  0 | q2 toggles (0→1)
5   | 0  1  0  1 |
6   | 0  1  1  0 |
7   | 0  1  1  1 | q2 stays 1
8   | 1  0  0  0 | q2 toggles (1→0)
```

Pattern: q2 toggles when both q1 AND q0 are 1

So what should **t2** be?

Question 3: NOT ANSWERED

You didn't answer this at all.

Question: Your 2-bit counter wraps from 3 → 0. In a 4-bit counter, what value would cause overflow (wrap to 0)?

Think:

- 2-bit counter: counts 0, 1, 2, 3 → wraps at 3 (binary: 11)
- 4-bit counter: counts 0, 1, 2, ..., ?, ? → wraps at ? (binary: 1111)

What's the maximum value a 4-bit counter can hold?

Retry the Questions

Question 2 (Clarified):

In a 4-bit binary counter:

```
assign t0 = 1'b1;           // Always toggle
assign t1 = q[0];           // Toggle when q[0]=1
assign t2 = ???;            // Fill this in
assign t3 = ???;            // Fill this in
```

What logic expressions should **t2 and **t3** be?**

Hint:

- **t2:** When should bit 2 flip? (Look at the pattern above)
- **t3:** When should bit 3 flip?

Question 3 (Restated):

A 4-bit counter can represent values from **0 to ?**

When it reaches this maximum value and increments again, it wraps to 0. What is this maximum value in:

- **Decimal:** ?
 - **Binary:** ????
-

Format Your Answer Like This:

QUESTION 2:

```
t2 = [your logic expression using q0, q1]
```

```
t3 = [your logic expression using q0, q1, q2]
```

Explanation: Bit 2 toggles when [explain the condition]

Bit 3 toggles when [explain the condition]

QUESTION 3:

Maximum value (decimal): [number]

Maximum value (binary): [4-bit binary]

Explanation: A 4-bit number can represent $2^4 = [?]$ different values...

Day 3 Started - JK Flip-Flop

Good question about master-slave flip-flops. Let me address that first, then start the lesson.

Master-Slave Flip-Flop Coverage

Short answer: Yes, but as a concept, not a separate implementation.

Why:

- **Historical context:** Master-slave was the OLD way to prevent timing issues (race conditions) in flip-flops
- **Modern reality:** Edge-triggered flip-flops (what we've been building) have **replaced** master-slave in virtually all modern designs
- **Synthesis tools:** When you write `always @(posedge clk)`, synthesizers already create proper edge-triggered logic—the master-slave problem is solved at the silicon level

When I'll explain it:

- During **JK Flip-Flop** lesson (today), I'll explain why the naive JK implementation has a **race condition**
- I'll show you the **master-slave solution** conceptually
- I'll explain why modern edge-triggered designs don't need explicit master-slave coding
- You'll understand it's a **hardware implementation detail**, not something you code in RTL

Bottom line: You need to **understand** the concept (important for interviews and debugging), but you won't **code** it separately because edge-triggered always blocks already solve the problem.

Day 3, Lesson 1: JK Flip-Flop

Concept Block (12 min)

What is a JK Flip-Flop?

JK = "Jack-Kilby" or "Jump-Kill" (disputed origin)

The most versatile flip-flop - can do everything:

- **Hold** current state
- **Reset** (force to 0)
- **Set** (force to 1)
- **Toggle** (flip state)

Truth Table:

J	K	Q(current)	Q(next)	Behavior
0	0	0	0	Hold
0	0	1	1	Hold
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	1	Toggle
1	1	1	0	Toggle

Simplified Truth Table:

J	K	Action
0	0	Hold ($q \leq q$)
0	1	Reset ($q \leq 0$)
1	0	Set ($q \leq 1$)
1	1	Toggle ($q \leq \sim q$)

Key insight: JK combines D, T, and SR flip-flop behaviors in one device!

Why Does This Exist?

Historical perspective:

- JK was the **universal flip-flop** in discrete logic (7476 IC)
- Could replace any other flip-flop type
- Reduced chip count in designs

Modern perspective:

- **Less common in RTL** - we usually write specific behavior (D or T)
- **Still used in:**
 - Legacy system maintenance
 - Educational contexts (understanding sequential logic)
 - Some control logic patterns
 - Interview questions (tests your understanding of sequential behavior)

Real-world usage:

- Modern FPGAs/ASICs implement D flip-flops in silicon
- You **emulate** JK behavior using D flip-flops when needed
- Direct JK implementation is rare in new designs

The Race Condition Problem (Master-Slave Context)

Naive JK implementation has a CRITICAL BUG:

Imagine this (WRONG) code:

```
// WRONG - DO NOT USE

always @(clk) begin // Sensitive to ANY change in clk (HIGH or LOW)
    if (j && !k)
        q <= 1;
    else if (!j && k)
        q <= 0;
    else if (j && k)
        q <= ~q;
end
```

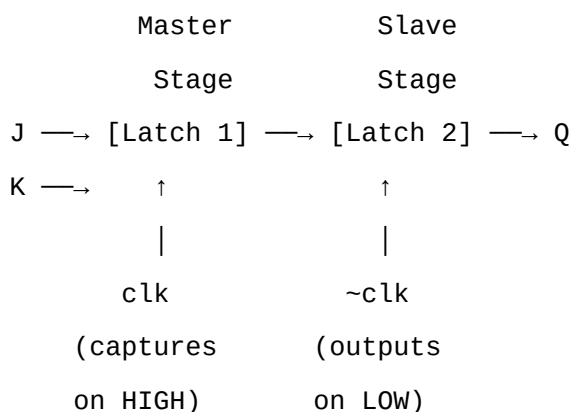
The problem:

- When J=K=1 (toggle mode)
- If clock is HIGH for any duration
- The always block keeps retriggering
- q toggles **multiple times** during the HIGH period
- Result: **unpredictable oscillation**

This is called a RACE CONDITION.

Solution 1: Master-Slave Architecture (Historical)

Concept:



How it works:

1. **Clock HIGH:** Master latch captures J, K inputs
2. **Clock LOW:** Slave latch copies master to output
3. **Result:** Output changes only ONCE per clock cycle

Why we don't code this:

- Modern synthesis tools handle this automatically

- Edge-triggered designs (`@(posedge clk)`) solve the problem
- Explicit master-slave coding is verbose and unnecessary

Solution 2: Edge-Triggered (Modern - What We Use)

```
// CORRECT - Modern approach
always @(posedge clk) begin // Triggers ONLY on rising edge
    case ({j, k})
        2'b00: q <= q;      // Hold
        2'b01: q <= 0;      // Reset
        2'b10: q <= 1;      // Set
        2'b11: q <= ~q;     // Toggle
    endcase
end
```

Why this works:

- `@(posedge clk)` triggers **once** per cycle (at the rising edge instant)
- Even if `J=K=1`, the `always` block executes **only once**
- No oscillation, no race condition
- Synthesizes to proper edge-triggered flip-flop

Key takeaway: Using `@(posedge clk)` gives you the master-slave benefit without explicit coding.

Comparison: JK vs Other Flip-Flops

Feature	D FF	T FF	JK FF
Inputs	1 (D)	1 (T)	2 (J, K)
Modes	Capture	Toggle/Hold	Set/Reset/Toggle/Hold
Complexity	Simple	Simple	Moderate
Modern usage	Very common	Common (counters)	Rare (legacy)
Versatility	Low	Low	High

When to use each:

- **D FF:** General storage, registers, most applications
 - **T FF:** Counters, frequency dividers
 - **JK FF:** When you need all four modes, legacy designs, educational
-

Code & Practical Block (12 min)

Implementation: JK Flip-Flop

```
// jk_ff.v - JK Flip-Flop
module jk_ff (
    input wire clk,    // Clock signal
```

```

input  wire rst,      // Asynchronous reset (active HIGH)
input  wire j,        // J input (set when j=1, k=0)
input  wire k,        // K input (reset when j=0, k=1)
output reg  q         // Output
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;          // Reset: clear output
    else begin
        case ({j, k})       // Concatenate j and k into 2-bit value
            2'b00: q <= q;    // Hold: maintain current state
            2'b01: q <= 1'b0; // Reset: force to 0
            2'b10: q <= 1'b1; // Set: force to 1
            2'b11: q <= ~q;   // Toggle: invert current state
        endcase
    end
end

endmodule

```

Line-by-Line Explanation

Lines 2-7: Port declarations

- `clk`: Synchronization signal
- `rst`: Async reset (highest priority, clears `q` to 0)
- `j, k`: Control inputs that determine behavior
- `q`: Registered output

Line 9: `always @(posedge clk or posedge rst)`

- Edge-triggered behavior (solves race condition)
- Asynchronous reset capability

Lines 10-11: Reset logic

- Priority 1: When `rst=1`, force `q=0`

Line 13: `case ({j, k})`

- **Concatenation operator** `{}`: Combines `j` and `k` into 2-bit value
- `{1'b0, 1'b1} → 2'b01`
- Allows clean 4-way decision

Lines 14-17: Four behavioral modes

- 2'b00 (j=0, k=0): Hold current value
- 2'b01 (j=0, k=1): Reset to 0
- 2'b10 (j=1, k=0): Set to 1
- 2'b11 (j=1, k=1): Toggle (invert)

Why case statement instead of if-else:

- Cleaner syntax for multiple conditions
- More readable for truth table implementations
- Synthesizes to same hardware as if-else
- Personal preference—both work

Alternative Implementation (If-Else Style)

```
always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else if (j && !k)           // j=1, k=0: Set
        q <= 1'b1;
    else if (!j && k)          // j=0, k=1: Reset
        q <= 1'b0;
    else if (j && k)           // j=1, k=1: Toggle
        q <= ~q;
    else                       // j=0, k=0: Hold
        q <= q;
end
```

Both styles are correct. Case statement is more structured.

Testbench

```
// jk_ff_tb.v - Testbench for JK Flip-Flop
module jk_ff_tb;
    reg  clk;
    reg  rst;
    reg  j;
    reg  k;
    wire q;

    // Instantiate DUT
    jk_ff uut (
```



```

        .clk(clk),
        .rst(rst),
        .j(j),
        .k(k),
        .q(q)
    );

// Clock generator: 10ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    $dumpfile("jk_ff.vcd");
    $dumpvars(0, jk_ff_tb);

    $display("Time | clk | rst | j | k | q | Mode");
    $display("-----");
    $monitor("%4t | %b | %b | %b | %b | %b | %s",
        $time, clk, rst, j, k, q,
        (rst) ? "RESET" :
        ({j,k}==2'b00) ? "HOLD" :
        ({j,k}==2'b01) ? "RESET" :
        ({j,k}==2'b10) ? "SET" : "TOGGLE");

    // Initialize
    rst = 1; j = 0; k = 0;
    #10;
    rst = 0;
    #10;

    // Test 1: Hold mode (j=0, k=0)
    $display("\n--- Test 1: Hold Mode ---");
    j = 0; k = 0; #20;

```

```

// Test 2: Set mode (j=1, k=0)
$display("\n--- Test 2: Set Mode ---");
j = 1; k = 0; #20;

// Test 3: Hold mode while q=1
$display("\n--- Test 3: Hold While Q=1 ---");
j = 0; k = 0; #20;

// Test 4: Reset mode (j=0, k=1)
$display("\n--- Test 4: Reset Mode ---");
j = 0; k = 1; #20;

// Test 5: Toggle mode (j=1, k=1)
$display("\n--- Test 5: Toggle Mode ---");
j = 1; k = 1; #40; // Multiple toggles

// Test 6: Async reset during operation
$display("\n--- Test 6: Async Reset ---");
j = 1; k = 0; #5; // Mid-cycle
rst = 1; #10;
rst = 0; #10;

$finish;
end

endmodule

```

Expected Simulation Output

Time	clk	rst	j	k	q	Mode	
0	0	1	0	0	0	RESET	
10	0	0	0	0	0	HOLD	
20	0	0	0	0	0	HOLD	(holding at 0)
30	0	1	1	0	0	SET	
40	0	1	1	0	1	SET	(q goes to 1)

50		0		0		0		0		1		HOLD	(holding at 1)
60		0		0		0		0		1		HOLD	
70		0		0		0		1		1		RESET	
80		0		0		0		1		0		RESET	(q goes to 0)
90		0		1		1		1		0		TOGGLE	
100		0		1		1		1		1		TOGGLE	(q toggles to 1)
110		0		1		1		1		0		TOGGLE	(q toggles to 0)
120		0		1		1		1		1		TOGGLE	(q toggles to 1)

Compile and Test

```
iverilog -o jk_ff jk_ff.v jk_ff_tb.v
vvp jk_ff
gtkwave jk_ff.vcd
```

Mini Task (5-8 min)

Task: Implement a **JK Flip-Flop behaving as a T Flip-Flop**

Specifications:

1. Use the `jk_ff` module you just created
2. Create a wrapper module that has only a `t` input (no `j`, `k`)
3. Wire `j` and `k` internally so that:
 - When `t=0`: Hold
 - When `t=1`: Toggle

Module signature:

```
module t_ff_from_jk (
    input  wire clk,
    input  wire rst,
    input  wire t,      // Toggle input
    output wire q
);
    // Your code here:
    // 1. Declare internal j, k wires
    // 2. Generate j, k from t
    // 3. Instantiate jk_ff
endmodule
```

Hint: Look at the JK truth table. What values of `j` and `k` give you toggle behavior?
